How does a Computer really work? *from Turing Machine to a real implemented custom 32Bits CPU*

Step 1: What is Computation?

Computation is the process of solving problems by executing a **finite sequence of well-defined operations**. At its core, any computation transforms input data into output data through a series of elementary steps.

A fundamental insight of computer science is that complex behavior can emerge from the repeated application of very simple operations. This observation motivates formal models of computation.

Step 2: The Turing Machine – A Formal Model of Computation

One of the most important theoretical models of computation is the **Turing Machine**. It consists of:

- A finite control (a **finite state machine**),
- A read/write head,
- An unbounded memory tape.

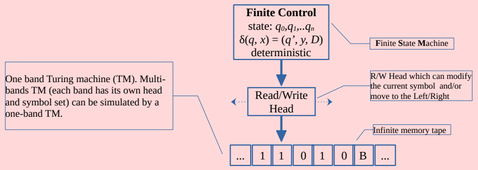
The behavior of the machine is defined by a transition function

$$\delta(q, x) = (q', y, D)$$

which specifies the next state, the symbol to write, and the direction to move.

Despite its simplicity, the Turing Machine can express **any algorithm**. This leads to a powerful conclusion:

Modern computers are not fundamentally more powerful than a Turing Machine - they are optimized physical realizations of the same computational idea.

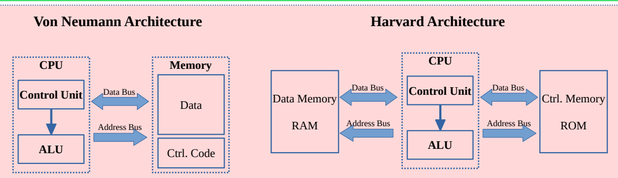


Step 3: Computer Architecture – Structuring Computation

To turn abstract computation into a real machine, we introduce computer architecture. Architecture defines how control, data, and memory are organized and how programs are executed.

- Von Neumann architecture: instructions and data share one memory
- Harvard architecture: instructions and data use separate memories

Architecture bridges computation theory and hardware realization.



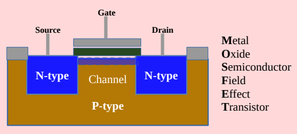
Step 4: From Physics to Logic

Physical computers are built from **transistors**, which act as electrically controlled switches. A transistor can either:

- allow current to flow, or
- block current.

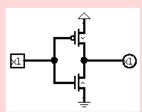
In digital systems, binary information is represented by two distinct voltage levels, commonly interpreted as logic **'1'** and logic **'0'**. Modern systems use **CMOS technology**, combining PMOS and NMOS transistors to achieve:

- low power consumption,
- stable switching behavior,
- high integration density.



N-type MOSFET

Metal
Oxide
Semiconductor
Field
Effect
Transistor

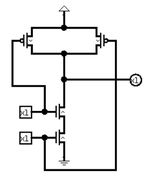


Simplest CMOS Pair acts as a NOT-Gate

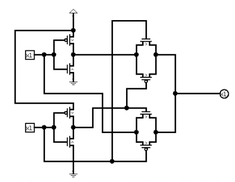
Step 5: Logic Gates and Universality

By combining transistors, we build **logic gates** such as AND, OR, and NOT. These gates operate on binary values and form the basis of all digital systems.

Some gates, such as **NAND** and **NOR**, are *universal*: any digital circuit can be constructed using only one of them. This demonstrates the power of abstraction in digital design.



Example 1: a static (pull-up/-down network) CMOS based NAND Gate



Example 2: a dynamic (Transmission Gates) CMOS based XOR Gate

Logic Function	NAND-only logic form	NAND count
NOT A	A NAND A	1
A AND B	(A NAND B) NAND (A NAND B)	2
A OR B	(ANAND A) NAND (B NAND B)	3
A XOR B	(A NAND (A NAND B)) NAND (B NAND (A NAND B))	4*

** one NAND Gate will be reused, there is only 4 but not 5*

Example 3: NAND as universal logic gate

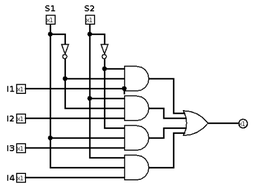
Step 6: Combinational Logic - Building the Datapath

Combinational logic circuits are formed by combining logic gates without memory elements. Their key properties are:

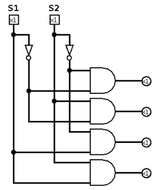
- outputs depend only on current inputs,
- no internal state,
- no clock signal.

Typical examples include:multiplexers, demultiplexers, encoders, decoders, adders and comparators.

These components form the datapath of a CPU — the part that moves and transforms data.



Example 1: a 4x1 multiplexer based on simple gates



Example 2: a 2x4 decoder based on simple gates

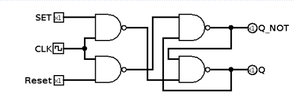
Step 7: Sequential Logic: Remembering State

Sequential logic extends combinational logic by introducing memory. Its properties are:

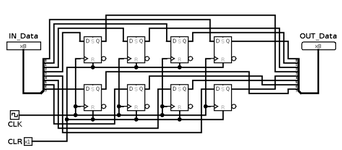
- outputs depend on current inputs and previous state,
- clocked operation,
- explicit storage elements.

Examples include flip-flops, registers, counters and memory cells.

Sequential logic allows a system to remember information over time, which is essential for program execution.



Example 1: a SR-FlipFlop based on NAND Gates

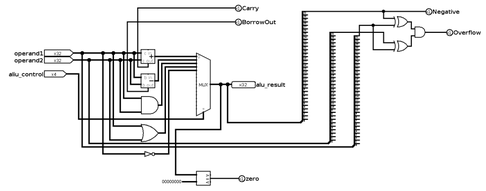


Example 2: an 8Bits Register based on 8 DFlipFlops

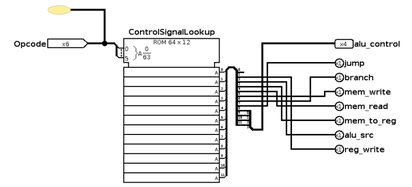
Step 8: Core CPU Components

A CPU is composed of several key building blocks:

- Program Counter (PC) – holds the address of the current instruction and determines the next instruction address (e.g. PC+1, branch, jump)
- Register File – fast local storage for operands,
- ALU (Arithmetic Logic Unit) – performs arithmetic and logical operations,
- Control Unit – generates control signals based on the current instruction,
- Memory Interfaces – handle communication with instruction and data memory.



Example 1: a custom ALU with different OPs



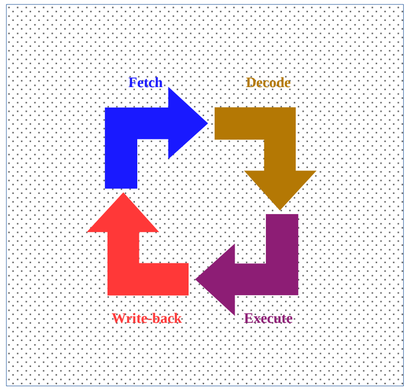
Example 2: a custom ROM-based Control Unit

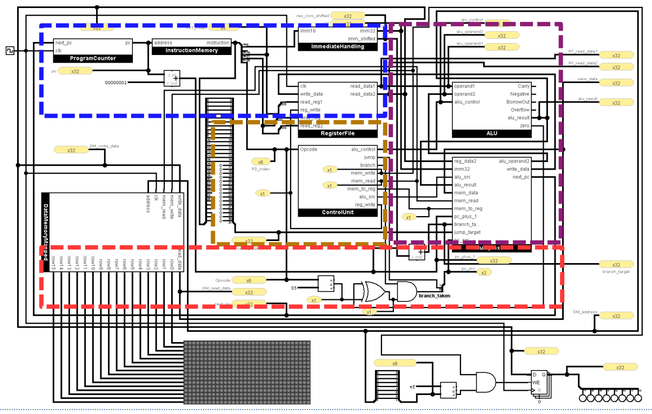
Step 9: How a CPU Executes Programs

A CPU operates in a repeating instruction cycle:

1. Fetch – read instruction from instruction memory,
2. Decode – interpret the opcode and operands,
3. Execute – perform the operation in the datapath,
4. Write-back – store the result.

This cycle executes continuously, advancing program state step by step.





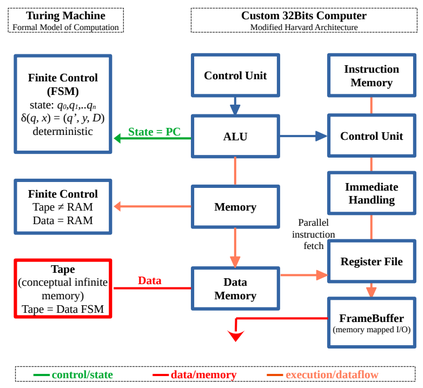
Final Step: Back to Theory: Validation

We now return to the original theoretical question: does the implemented system conform to the theory of computation?
Because the system has:

- finite, deterministic control,
- conditional branching,
- and memory that is expandable in principle,

☑ It is computationally equivalent to a Turing Machine.

The loop - **theory → architecture → hardware → theory** - is closed. What begins as an abstract model of computation ends as a real-working computer system.



Usage Guideline

This poster is designed to be read in sequence from Step 1 to the Final Step.

The left side introduces the theory of computation and computer architecture.

The center explains how logic and memory are built from hardware primitives.
The right side connects these ideas to CPU execution and a real implemented system.

Readers may also jump directly to individual sections depending on their background.

Downlaod link: https://github.com/ywinf/Big-Pictures